Estructuras de Datos y Algoritmos

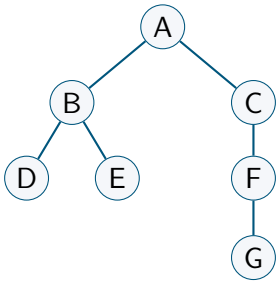
Tema 16: Grafos

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Grafos

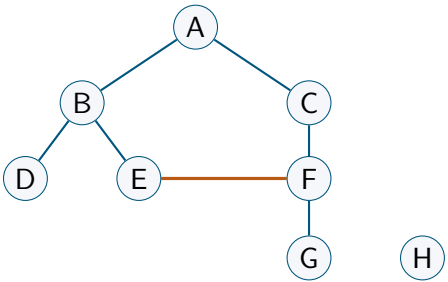
Jerarquía



Desde la raíz hay un único camino hasta cada nodo.

Un árbol es un caso particular de grafo.

Red de relaciones

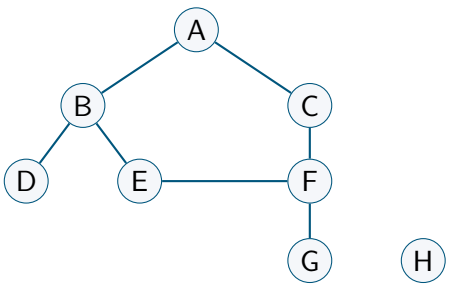


Puede haber ciclos, varios caminos entre dos vértices y vértices aislados.

Grafo

Definición: Grafo

Un grafo finito $G = (V, E)$ está formado por un conjunto de vértices V y un conjunto de aristas E, que representan relaciones entre ellos.



$$V = \{A, B, C, D, E, F, G, H\}.$$

Las aristas del ejemplo son:

$$\{A, B\},\ \{A, C\},\ \{B, D\},\ \{B, E\},$$
$$\{C, F\},\ \{E, F\},\ \{F, G\}.$$

$$\boxed{n = |V| = 8, \qquad m = |E| = 7.}$$

Un vértice también se llama **nodo**. No hay una raíz.

Dirigidos y no dirigidos

Grafo no dirigido

Una arista no dirigida $\{u, v\}$ relaciona sus extremos sin asignarles una dirección.

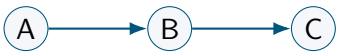


$$E = \{\{A, B\}, \{B, C\}\}.$$

La conexión entre A y B puede recorrerse en ambos sentidos.

Grafo dirigido

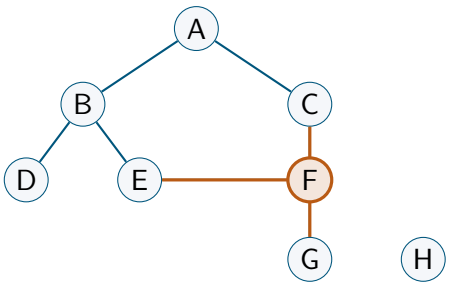
Una arista dirigida (u, v) tiene origen u y destino v; el orden importa.



$$E = \{(A, B), (B, C)\}.$$

Una arista dirigida de A a B no implica que exista otra de B a A.

Adyacencia y grado



Adyacencia

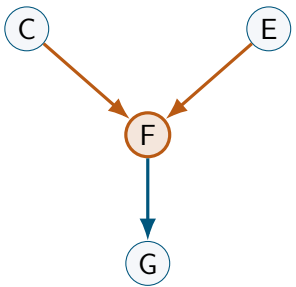
Dos vértices son adyacentes si una arista los une.

Grado de un vértice

En este grafo no dirigido, $\deg(v)$ cuenta las aristas incidentes en v.

$$\deg(F) = 3, \qquad \deg(H) = 0.$$

Grado de entrada y salida



Grado de entrada

Cantidad de aristas dirigidas que llegan al vértice.

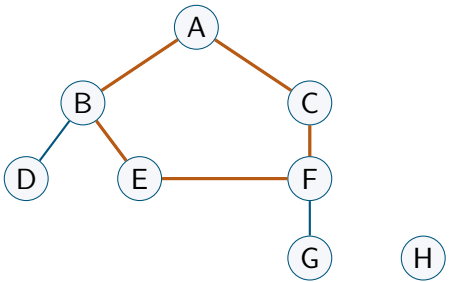
$C \to F, \quad E \to F \quad \Longrightarrow \quad \deg^-(F) = 2.$

Grado de salida

Cantidad de aristas dirigidas que salen del vértice.

$F \to G \quad \Longrightarrow \quad \deg^+(F) = 1.$

Caminos y ciclos



Camino

Secuencia de vértices distintos con una arista entre cada par consecutivo. En un grafo dirigido se respeta la dirección de sus aristas.
La **longitud** es el número de aristas: para $P = (v_0, \ldots, v_k)$, es k.

D, B, E, F, G es un camino de **longitud 4**.

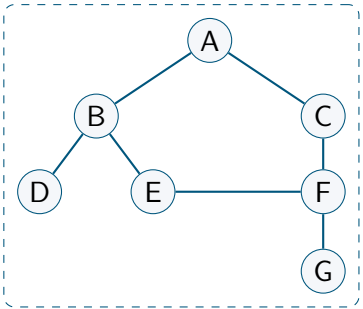
Ciclo

Recorrido de aristas distintas que vuelve al vértice inicial sin repetir los demás vértices.

El ciclo resaltado es A, B, E, F, C, A.

Sin lazos: un ciclo tiene al menos tres aristas en un grafo no dirigido, o dos en uno dirigido.

Conectividad no dirigida



En un grafo no dirigido, dos vértices son **alcanzables entre sí** si existe un camino que los conecta.

Grafo conexo

Un grafo no dirigido es conexo si hay un camino entre cualquier par de vértices.

Componente conexa

Conjunto maximal de vértices conectados mediante caminos.

En el ejemplo hay dos componentes:

$$\{A, B, C, D, E, F, G\}, \qquad \{H\}.$$

H está **aislado** y forma una componente por sí solo. El grafo del ejemplo no es conexo.

Conectividad dirigida

En un grafo dirigido, los caminos deben respetar la dirección de las aristas.

Débilmente conexo

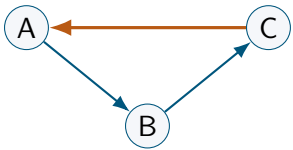
Al ignorar la dirección de sus aristas, el grafo no dirigido resultante es conexo.



Es débilmente conexo, pero **no fuertemente conexo**: no hay un camino de C a A.

Fuertemente conexo

Desde cualquier vértice se puede llegar a cualquier otro respetando las direcciones.



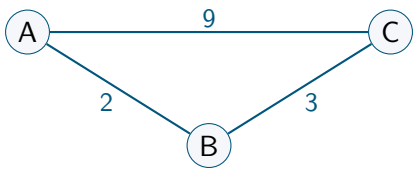
Los tres vértices son **mutuamente alcanzables**.

Todo grafo fuertemente conexo también es débilmente conexo.

Pesos

Peso de una arista

Valor asociado a una conexión: puede representar distancia, tiempo o costo. El peso de la arista es distinto del dato almacenado en el vértice.



Camino A, C: longitud 1; costo 9.

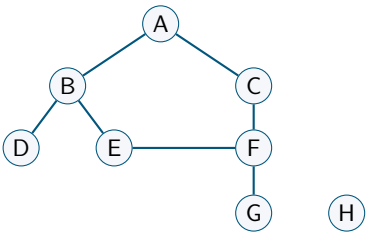
Camino A, B, C: longitud 2; costo $2 + 3 = 5$.

Menos aristas no significa necesariamente menor costo.

Sea $w(u, v)$ el peso de la arista entre los vértices u y v. Si $P = (v_0, \ldots, v_k)$, su costo es $C(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$. En un grafo ponderado, los pesos pueden ser cero.

Matriz de adyacencia

Cada vértice tiene un índice. La fila indica el origen y la columna, el destino.

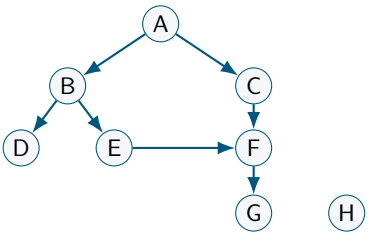


	A	B	C	D	E	F	G	H
A	0	1	1	0	0	0	0	0
B	1	0	0	1	1	0	0	0
C	1	0	0	0	0	1	0	0
D	0	1	0	0	0	0	0	0
E	0	1	0	0	0	1	0	0
F	0	0	1	0	1	0	1	0
G	0	0	0	0	0	1	0	0
H	0	0	0	0	0	0	0	0

1 indica arista y 0, ausencia de arista. La matriz es simétrica porque el grafo no es dirigido.

Matriz de adyacencia: grafo dirigido

Cada vértice tiene un índice. La fila indica el origen y la columna, el destino.

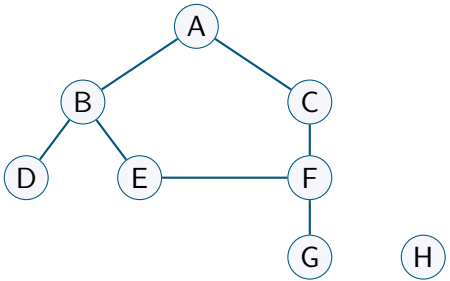


	A	B	C	D	E	F	G	H
A	0	1	1	0	0	0	0	0
B	0	0	0	1	1	0	0	0
C	0	0	0	0	0	1	0	0
D	0	0	0	0	0	0	0	0
E	0	0	0	0	0	1	0	0
F	0	0	0	0	0	0	1	0
G	0	0	0	0	0	0	0	0
H	0	0	0	0	0	0	0	0

1 indica arista del origen al destino y 0, ausencia de arista. La matriz no es simétrica.

Listas de adyacencia

Cada vértice guarda pares (destino, peso). En este ejemplo, todos los pesos son 1.

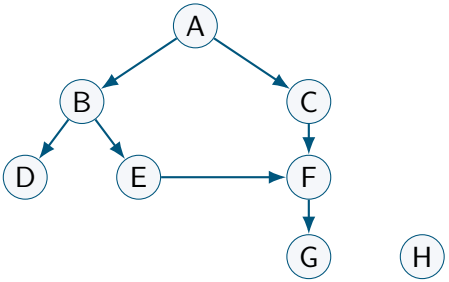


Vértice	Adyacentes
A	`[(B, 1), (C, 1)]`
B	`[(A, 1), (D, 1), (E, 1)]`
C	`[(A, 1), (F, 1)]`
D	`[(B, 1)]`
E	`[(B, 1), (F, 1)]`
F	`[(C, 1), (E, 1), (G, 1)]`
G	`[(F, 1)]`
H	`[]`

Cada arista no dirigida aparece en dos listas: hay $2m = 14$ entradas.

Listas de adyacencia: grafo dirigido

Cada vértice guarda sus aristas de salida como pares (destino, peso). Todos los pesos son 1.



Vértice	Adyacentes de salida
A	`[(B, 1), (C, 1)]`
B	`[(D, 1), (E, 1)]`
C	`[(F, 1)]`
D	`[]`
E	`[(F, 1)]`
F	`[(G, 1)]`
G	`[]`
H	`[]`

Cada arista dirigida aparece solo en la lista de su origen: hay $m = 7$ entradas. D y G tienen grado de salida cero, aunque reciben aristas. H está aislado.

Comparar representaciones

Matriz

Reserva una posición para cada par de vértices, exista o no una arista.

Con sus índices conocidos, podemos consultar directamente esa posición.

Listas de adyacencia

Cada vértice guarda una lista de sus conexiones existentes (de salida, si el grafo es dirigido).

Para buscar un destino, revisamos las entradas de la lista del origen.

TDA grafo

Una interfaz de grafo puede incluir las siguientes operaciones:

Métodos	Propósito
`insertar_vertice`, `eliminar_vertice`	Agregar o retirar un vértice.
`buscar_vertice`	Localizar un vértice por su dato.
`insertar_arista`, `eliminar_arista`	Agregar o retirar una arista.
`consultar_adyacentes`	Consultar vecinos y pesos; destinos de salida si es dirigido.
`profundidad`, `amplitud`	Recorrer los vértices alcanzables desde un origen.
`dijkstra`	Calcular distancias mínimas con pesos no negativos.
`camino`	Reconstruir un camino a partir de una búsqueda.

En esta clase solo desarrollaremos algunas operaciones esenciales para representar el grafo, recorrerlo y encontrar caminos.

Representación en Python

```
class NodoVertice:
    def __init__(self, dato):
        self.dato = dato
        self.adyacentes = []
        self.visitado = False
```

`visitado` es una marca temporal de exploración.

```
class Grafo:
    def __init__(self, dirigido=False):
        self.vertices = []
        self.dirigido = dirigido

    def insertar_vertice(self, nodo):
        self.vertices.append(nodo)
```

Registrar un nodo nuevo cuesta $\Theta(1)$ amortizado; cada nodo se registra una sola vez.

Cada entrada de `adyacentes` será un par $(\text{destino}, \text{peso})$.

Conectar vértices

Método InsertarArista
(origen, destino, peso)

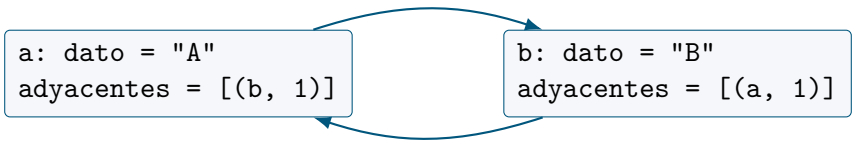
Añadir (destino, peso) a origen.adyacentes

Si self.dirigido = falso:
    Añadir (origen, peso) a destino.adyacentes

```
def insertar_arista(self, origen,
                    destino,
                    peso=1):
    origen.adyacentes.append(
        (destino, peso))
    if not self.dirigido:
        destino.adyacentes.append(
            (origen, peso))
```

`append` añade el par al final de la lista.

Ejemplo no dirigido, con listas vacías: `grafo.insertar_arista(a, b)`.



Costo: $\Theta(1)$.

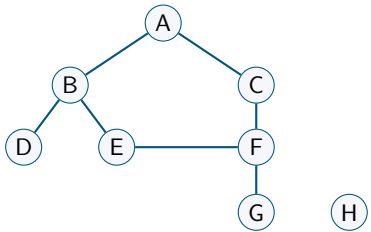
Construir un grafo

Crear y registrar los ocho nodos.

```
grafo = Grafo()
a = NodoVertice("A")
b = NodoVertice("B")
c = NodoVertice("C")
d = NodoVertice("D")
e = NodoVertice("E")
f = NodoVertice("F")
g = NodoVertice("G")
h = NodoVertice("H")
for nodo in [a, b, c, d, e, f, g, h]:
    grafo.insertar_vertice(nodo)
```

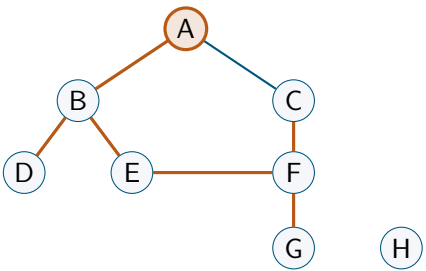
Después, agregar las siete aristas.

```
for origen, destino in [
    (a, b), (a, c), (b, d), (b, e),
    (c, f), (e, f), (f, g)
]:
    grafo.insertar_arista(origen,
    destino)
```



En profundidad

Elegimos un vértice del grafo como **origen** del recorrido. En este ejemplo comenzamos en **A** y exploramos los vértices alcanzables desde él.



DFS: búsqueda en profundidad (Depth-First Search).

Desde el origen, exploramos un vecino no visitado y continuamos desde él. Cuando no quedan vecinos pendientes, regresamos a la llamada anterior.

La recursión conserva el trabajo pendiente en la **pila de llamadas**.

El recorrido depende del origen y del orden de sus listas de adyacencia.

Algoritmo de DFS

`_reiniciar()` pone en falso la marca de todos los vértices antes de comenzar.

Auxiliar _profundidad(nodo, resultado)

nodo.visitado ← verdadero
Añadir nodo.dato a resultado
Para cada (vecino, peso) de nodo.adyacentes:
    Si vecino no está visitado:
        `_profundidad(vecino, resultado)`

Método Profundidad(origen)

`_reiniciar()`
resultado ← lista vacía
`_profundidad(origen, resultado)`
Retornar resultado

Visitar consiste aquí en añadir el dato a la lista de salida.

El auxiliar recursivo recibe directamente la referencia a cada vecino.

DFS en Python

El origen es un vértice registrado.

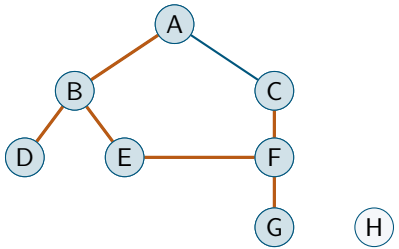
```
def _reiniciar(self):
    for nodo in self.vertices:
        nodo.visitado = False

def _profundidad(self, nodo, resultado):
    nodo.visitado = True
    resultado.append(nodo.dato)
    for vecino, peso in nodo.adyacentes:
        if not vecino.visitado:
            self._profundidad(
                vecino, resultado
            )
```

```
def profundidad(self, origen):
    self._reiniciar()
    resultado = []
    self._profundidad(
        origen, resultado
    )
    return resultado
```

`grafo.profundidad(a)` devuelve los **datos alcanzables**, sin modificar aristas.

Traza de DFS



Tras terminar D, regresamos a B. Tras terminar C, regresamos a F.

Se descubre	Llamadas activas
A	A
B	A, B
D	A, B, D
E	A, B, E
F	A, B, E, F
C	A, B, E, F, C
G	A, B, E, F, G

Desde C no volvemos a A: ya está marcado.

Salida: `A, B, D, E, F, C, G`.
H no es alcanzable desde A.

Costo de DFS

Sean $n = |V|$, $m = |E|$, r la cantidad de vértices alcanzables y a_R la cantidad de entradas en sus listas de adyacencia.

- Reiniciar marcas recorre los n vértices.
- Cada vértice alcanzable se procesa una sola vez: r visitas.
- Cada entrada de sus listas se examina una vez: a_R comprobaciones.

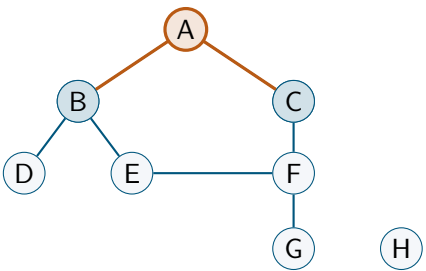
$$T(n, m) \leq an + br + ca_R + d.$$

Para un grafo no dirigido, $r \leq n$ y

$$a_R \leq \sum_{v \in V} |v.\text{adyacentes}| = 2m \quad \Longrightarrow \quad T(n, m) \leq (a + b)n + 2cm + d.$$

Por tanto, $\boxed{T(n, m) \in O(n + m)}$. En un dirigido, la suma global es m, y se obtiene la misma cota.

En amplitud



BFS: búsqueda en amplitud (Breadth-First Search).

Una cola conserva los vértices descubiertos que todavía debemos procesar.

Desde A, primero se descubren B y C. **FIFO** permite procesarlos antes que los vértices descubiertos desde ellos.

Como en el recorrido por niveles, usamos una cola. Ahora marcamos cada vértice **al encolarlo**, para no añadirlo varias veces.

Algoritmo de BFS

Auxiliar _amplitud(origen, resultado)

$Q \leftarrow$ cola vacía
origen.visitado $\leftarrow$ verdadero
Encolar(Q, origen)
Mientras Q no esté vacía:
    nodo $\leftarrow$ Desencolar(Q)
    Añadir nodo.dato a resultado
    Para cada (vecino, peso) de nodo.adyacentes:
        Si vecino no está visitado:
            vecino.visitado $\leftarrow$ verdadero
            Encolar(Q, vecino)

Método Amplitud(origen)

```
_reiniciar()
```
resultado $\leftarrow$ lista vacía
```
_amplitud(origen, resultado)
```
Retornar resultado

Como en DFS, el método principal reinicia las marcas **una sola vez**.

El auxiliar explora con una cola y marca cada vecino **al encolarlo**.

BFS en Python

Usamos `from collections import deque`.

```
def _amplitud(self, origen,
              resultado):
    origen.visitado = True
    pendientes = deque([origen])
    while pendientes:
        nodo = pendientes.popleft()
        resultado.append(nodo.dato)
        for vecino, peso in nodo.adyacentes:
            if not vecino.visitado:
                vecino.visitado = True
                pendientes.append(vecino)
```

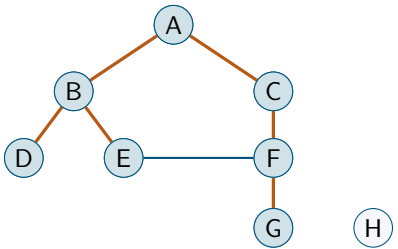
```
def amplitud(self, origen):
    self._reiniciar()
    resultado = []
    self._amplitud(
        origen, resultado
    )
    return resultado
```

Se reutiliza `_reiniciar()` de DFS. La cola conserva nodos; `resultado`, sus datos.

Costo: $O(n + m)$, incluido el reinicio; mismo conteo de vértices y adyacencias que en DFS.

Traza de BFS

Inicio: cola `[A]` y A marcado.



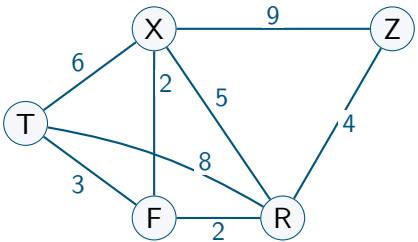
Al procesar E, F ya estaba marcado y encolado: C lo descubrió antes.

Procesado	Cola al terminar	Nuevos
A	`[B, C]`	B, C
B	`[C, D, E]`	D, E
C	`[D, E, F]`	F
D	`[E, F]`	—
E	`[F]`	—
F	`[G]`	G
G	`[]`	—

Salida: `A, B, C, D, E, F, G`.

H queda sin visitar: no es alcanzable desde A.

Dijkstra



1. Asignar 0 al origen e ∞ a los demás. Todos comienzan sin fijar.
2. Elegir el pendiente de **menor distancia provisional**.
3. Fijar su distancia: ya es definitiva.
4. Para cada vecino pendiente, calcular **distancia del elegido + peso de la arista**. Si la suma es menor que la distancia del vecino, actualizar esta última.
5. Repetir desde el paso 2 hasta que no queden pendientes con distancia finita.

Datos de Dijkstra

Para Dijkstra añadimos tres atributos a cada vértice.

distancia: costo del mejor camino encontrado hasta el momento desde el origen.

anterior: referencia al vértice previo en ese camino; permite reconstruirlo.

fijado: indica que la distancia almacenada ya es definitiva.

Dentro de `__init__` de `NodoVertice`, añadimos:

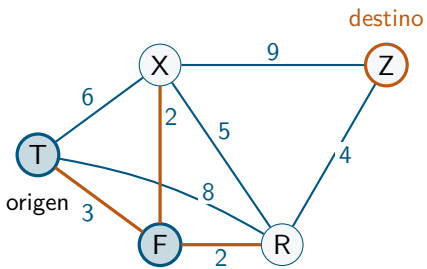
```
self.distancia = float("inf")
self.anterior = None
self.fijado = False
```

Dijkstra reinicia estos tres atributos en cada ejecución y asigna 0 a la distancia del origen.

Un vértice pendiente tiene `fijado = False`, aunque ya tenga una distancia finita.

Seguimiento

Origen: T. Destino que nos interesa: Z. Dijkstra calcula las distancias desde T hacia los vértices alcanzables.



Se fija	Distancia	Cambios en pendientes
T	0	$F:3,\ X:6,\ R:8$
F	3	$X:6\to 5,\ R:8\to 5$

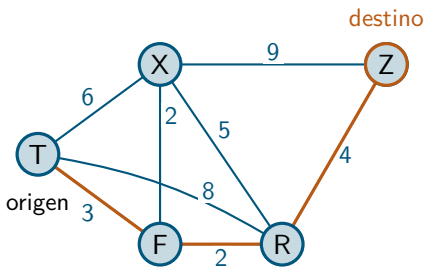
Elegimos F porque 3 es la menor distancia pendiente.

Desde F, llegar a X o a R cuesta $3+2=5$. Sus distancias mejoran, pero ambos siguen pendientes.

Z todavía no tiene un camino conocido: distancia ∞ y `fijado = False`.

Seguimiento

X y R empatan con distancia 5. Elegimos X primero por el orden de registro.



Se fija	Distancia	Distancia de Z
X	5	$5 + 9 = 14$, provisional
R	5	$5 + 4 = 9$, provisional
Z	9	**9, definitiva**

Al mejorar a 9, el anterior de Z cambia de X a R.

Solo al elegir Z confirmamos su distancia y ponemos su `fijado = True`.

Camino mínimo hasta Z: $T \to F \to R \to Z$, de costo $3 + 2 + 4 = 9$.

Algoritmo de Dijkstra

Método Dijkstra(origen)

```
Para cada vertice en vertices:
    vertice.fijado ← falso
    vertice.distancia ← ∞
    vertice.anterior ← vacío
origen.distancia ← 0
Repetir |vertices| veces:
    nodo ← vacío
    Para cada candidato en vertices:
        Si no candidato.fijado:
            Si nodo es vacío o
                candidato.distancia
                 < nodo.distancia:
                nodo ← candidato
```

Continuación del ciclo exterior

```
    Si nodo es vacío o
        nodo.distancia = ∞:
        Terminar el ciclo exterior
    nodo.fijado ← verdadero
    Para cada (vecino, peso) en
        nodo.adyacentes:
        Si no vecino.fijado:
            nueva ← nodo.distancia + peso
            Si nueva < vecino.distancia:
                vecino.distancia ← nueva
                vecino.anterior ← nodo
```

Dijkstra en Python

```
def dijkstra(self, origen):
    for vertice in self.vertices:
        vertice.fijado = False
        vertice.distancia = float("inf")
        vertice.anterior = None
    origen.distancia = 0
    for _ in self.vertices:
        nodo = None
        for candidato in self.vertices:
            if not candidato.fijado:
                if (nodo is None or
                        candidato.distancia < nodo.distancia):
                    nodo = candidato
```

Cada selección recorre la lista completa de vértices. En un empate se conserva el primero encontrado.

Dijkstra en Python

Continuación: dentro del ciclo exterior, después de elegir `nodo`.

```
        if nodo is None or nodo.distancia == float("inf"):
            break
        nodo.fijado = True
        for vecino, peso in nodo.adyacentes:
            if not vecino.fijado:
                nueva = nodo.distancia + peso
                if nueva < vecino.distancia:
                    vecino.distancia = nueva
                    vecino.anterior = nodo
```

El resultado queda en `distancia` y `anterior` de cada nodo. El método devuelve `None`.

Costo de Dijkstra

Sean $n = |V| \geq 1$ y $m = |E|$.

- Inicializar `fijado`, `distancia` y `anterior` recorre los n nodos.
- Hay a lo sumo n selecciones; cada una examina n candidatos.
- Cada lista de adyacencia se recorre una sola vez al fijar su nodo.

$$\begin{aligned} T(n, m) &\leq an + b \sum_{i=1}^{n} n + cm \\ &= an + bn^2 + cm \\ &\in O(n^2 + m). \end{aligned}$$

En un grafo no dirigido, las $2m$ entradas de adyacencia se absorben en la constante c.
Si todos los vértices son alcanzables, se realizan las n selecciones completas: $\Omega(n^2)$. Para grafos simples, $m \leq n(n-1)$, y por tanto $\boxed{T_{\text{peor}}(n) \in \Theta(n^2)}$.

Reconstruir un camino

Dijkstra ejecutado desde este origen, sin cambiar el grafo.

Método Camino(origen, destino)

Si destino.distancia $= \infty$:
    Retornar lista vacía
resultado $\leftarrow$ lista vacía
actual $\leftarrow$ destino
Mientras actual no sea el nodo origen:
    Añadir actual.dato a resultado
    actual $\leftarrow$ actual.anterior
Añadir origen.dato a resultado
Invertir resultado
Retornar resultado

```
def camino(self, origen, destino):
    if (destino.distancia
            == float("inf")):
        return []
    resultado = []
    actual = destino
    while actual is not origen:
        resultado.append(actual.dato)
        actual = actual.anterior
    resultado.append(origen.dato)
    resultado.reverse()
    return resultado
```

Costo: $\Theta(\ell)$ para un camino de ℓ aristas; $\Theta(1)$ si el destino es inalcanzable.

Obtener el camino

El método `camino` sigue `anterior` desde el destino hasta el origen y revierte la secuencia.

Con `origen` en T y `destino` en Z:

```
grafo.dijkstra(origen)
grafo.camino(origen, destino)  # ['T', 'F', 'R', 'Z']
destino.distancia              # 9
```

- Origen igual a destino: un solo dato y distancia 0.
- Destino inalcanzable: [] y distancia ∞.